

BC Experiment-6

AIM: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

THEORY:

What is a Smart Contract? A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- **Automation:** Executes actions automatically when conditions are satisfied.
- **Transparency:** Contract code and transactions are visible on the blockchain.
- **Security:** Once deployed, contracts are immutable and tamper-proof.
- **Cost-effective:** Removes intermediaries and reduces transaction costs.
- **Trustless System:** Participants can interact without needing to trust each other directly.

Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

About the Crowdfunding Platform: A blockchain-based crowdfunding platform replaces centralized authorities with smart contracts. Two contracts are designed — **CrowdfundingContract.sol**, which manages campaign creation and tracks donations, and **DonorContract.sol**, which handles donor interactions and communicates with the CrowdfundingContract through a Solidity interface. Together they ensure a transparent, trustless, and intermediary-free fundraising system.

Tools Used:

- **Solidity** — High-level language for writing smart contracts on Ethereum.
- **Remix IDE** — Browser-based IDE for writing, compiling, deploying, and testing smart contracts using a built-in Remix VM.

IMPLEMENTATION:

CrowdfundingContract.sol

```
// SPDX-License-Identifier: GPL-3.0  
pragma solidity ^0.8.0;
```

```
contract CrowdfundingContract {
```

```

struct Campaign {
    uint256 campaignId;
    string title;
    uint256 goalAmount;
    uint256 raisedAmount;
    uint256 deadline;
    address payable creator;
    bool isActive;
}

uint256 public campaignCount = 0;
mapping(uint256 => Campaign) public campaigns;

    event CampaignCreated(uint256 campaignId, string title, uint256 goalAmount,
address creator);
    event FundsReceived(uint256 campaignId, uint256 amount);

function createCampaign(
    string memory _title,
    uint256 _goalAmount,
    uint256 _durationDays
) public {
    campaignCount++;
    uint256 deadline = block.timestamp + (_durationDays * 1 days);
    campaigns[campaignCount] = Campaign(
        campaignCount,
        _title,
        _goalAmount,
        0,
        deadline,
        payable(msg.sender),
        true
    );
    emit CampaignCreated(campaignCount, _title, _goalAmount, msg.sender);
}

function receiveDonation(uint256 _campaignId, uint256 _amount) external {
    Campaign storage c = campaigns[_campaignId];
    require(c.isActive, "Campaign is not active");
    c.raisedAmount += _amount;
    emit FundsReceived(_campaignId, _amount);
}

function getCampaign(uint256 _campaignId) external view returns (
    uint256, string memory, uint256, uint256, uint256, address, bool

```

```

    ){
        Campaign storage c = campaigns[_campaignId];
        return (
            c.campaignId,
            c.title,
            c.goalAmount,
            c.raisedAmount,
            c.deadline,
            c.creator,
            c.isActive
        );
    }

    function closeCampaign(uint256 _campaignId) external {
        campaigns[_campaignId].isActive = false;
    }
}

```

DonorContract.sol

// SPDX-License-Identifier: GPL-3.0

pragma solidity ^0.8.0;

```

interface ICrowdfundingContract {
    function receiveDonation(uint256 _campaignId, uint256 _amount) external;
    function getCampaign(uint256 _campaignId) external view returns (
        uint256, string memory, uint256, uint256, uint256, address, bool
    );
}

contract DonorContract {

    struct Donation {
        uint256 donationId;
        uint256 campaignId;
        address donor;
        uint256 amount;
        uint256 timestamp;
    }

    uint256 public donationCount = 0;
    mapping(uint256 => Donation) public donations;
    address public crowdfundingContractAddress;

    event DonationMade(uint256 donationId, uint256 campaignId, address donor, uint256
amount);
    constructor(address _crowdfundingAddress) {
        crowdfundingContractAddress = _crowdfundingAddress;
    }
}

```

```

function donate(uint256 _campaignId, uint256 _amount) public {
    donationCount++;
    donations[donationCount] = Donation(
        donationCount,
        _campaignId,
        msg.sender,
        _amount,
        block.timestamp
    );
    ICrowdfundingContract(crowdfundingContractAddress)
        .receiveDonation(_campaignId, _amount);

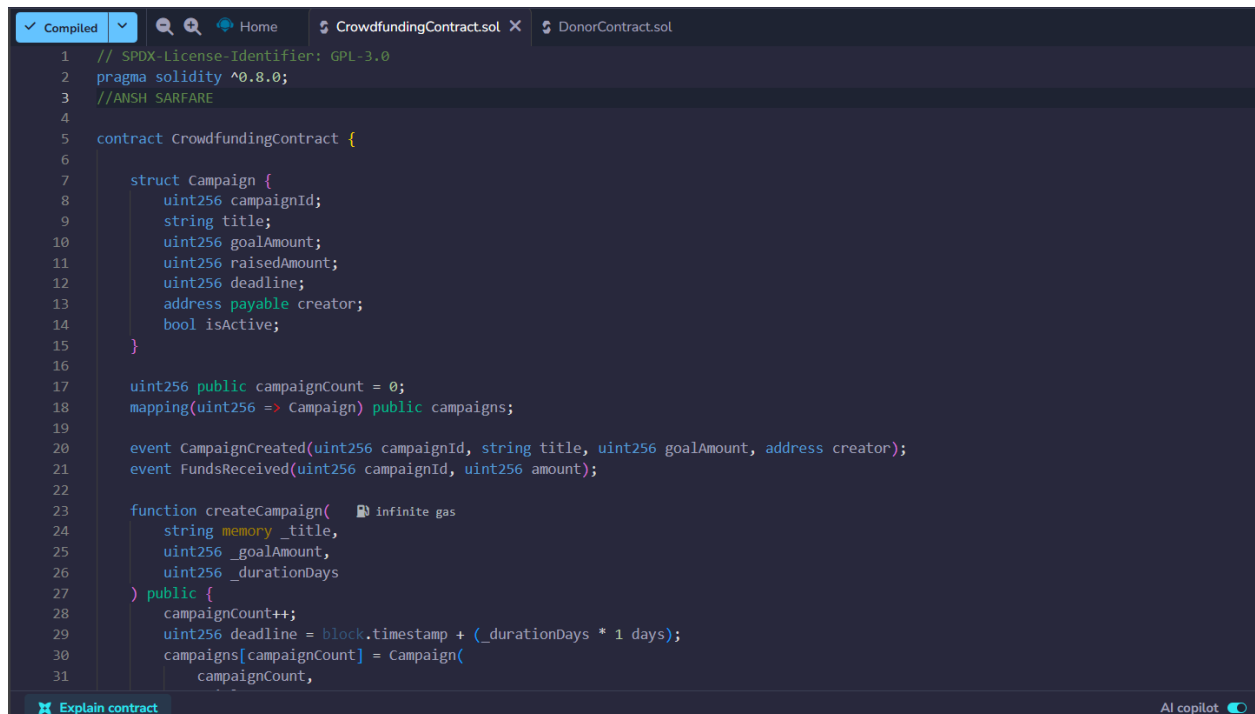
    emit DonationMade(donationCount, _campaignId, msg.sender, _amount);
}

function getDonation(uint256 _donationId) public view returns (
    uint256, uint256, address, uint256, uint256
) {
    Donation storage d = donations[_donationId];
    return (d.donationId, d.campaignId, d.donor, d.amount, d.timestamp);
}

function getCrowdfundingContract() public view returns (address) {
    return crowdfundingContractAddress;
}
}

```

Compilation:



```

1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.8.0;
3 //ANSH SARFARE
4
5 contract CrowdfundingContract {
6
7     struct Campaign {
8         uint256 campaignId;
9         string title;
10        uint256 goalAmount;
11        uint256 raisedAmount;
12        uint256 deadline;
13        address payable creator;
14        bool isActive;
15    }
16
17    uint256 public campaignCount = 0;
18    mapping(uint256 => Campaign) public campaigns;
19
20    event CampaignCreated(uint256 campaignId, string title, uint256 goalAmount, address creator);
21    event FundsReceived(uint256 campaignId, uint256 amount);
22
23    function createCampaign( Infinite gas
24        string memory _title,
25        uint256 _goalAmount,
26        uint256 _durationDays
27    ) public {
28        campaignCount++;
29        uint256 deadline = block.timestamp + (_durationDays * 1 days);
30        campaigns[campaignCount] = Campaign(
31            campaignCount,

```

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.8.0;
3 //ANSH SARFARE
4
5 interface ICrowdfundingContract {
6     function receiveDonation(uint256 _campaignId, uint256 _amount) external;
7     function getCampaign(uint256 _campaignId) external view returns (
8         uint256, string memory, uint256, uint256, uint256, address, bool
9     );
10 }
11
12 contract DonorContract {
13
14     struct Donation {
15         uint256 donationId;
16         uint256 campaignId;
17         address donor;
18         uint256 amount;
19         uint256 timestamp;
20     }
21
22     uint256 public donationCount = 0;
23     mapping(uint256 => Donation) public donations;
24     address public crowdfundingContractAddress;
25
26     event DonationMade(uint256 donationId, uint256 campaignId, address donor, uint256 amount);
27
28     constructor(address _crowdfundingAddress) {
29         crowdfundingContractAddress = _crowdfundingAddress;
30     }
31 }
```

DEPLOYMENT:

DEPLOY & RUN TRANSACTIONS

Environment

Remix VM

Ansh

0x5B3...eddC4

99.999 ETH

value

0

wei

Gas limit

0

Deploy

Deployed Contracts 2

CrowdfundingContract

0xd91...39138

Remix VM

5min ago

DonorContract

0xd8b...33fa8

Remix VM

40s ago

Contract Code

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.8.0;
3 //ANSH SARFARE
4
5 interface ICrowdfundingContract {
6     function receiveDonation(uint256 _campaignId, uint256 _amount) external;
7     function getCampaign(uint256 _campaignId) external view returns (
8         uint256, string memory, uint256, uint256, uint256, address, bool
9     );
10 }
11
12 contract DonorContract {
13
14     struct Donation {
15         uint256 donationId;
16         uint256 campaignId;
17         address donor;
18         uint256 amount;
19         uint256 timestamp;
20     }
21
22     uint256 public donationCount = 0;
23     mapping(uint256 => Donation) public donations;
24     address public crowdfundingContractAddress;
25
26     event DonationMade(uint256 donationId, uint256 campaignId, address donor, uint256 amount);
27
28     constructor(address _crowdfundingAddress) {
29         crowdfundingContractAddress = _crowdfundingAddress;
30     }
31 }
```

Explain contract

You can use this terminal to:

- Check transactions details and start debugging.
- Execute JavaScript scripts:
 - Input a script directly in the command line interface
 - Select a Javascript file in the file explorer and then run "remix.execute()" or "remix.executeCurrent()" in the command line interface
 - Right-click on a Javascript file in the file explorer and then click "Run"

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

[vm] from: 0x5B3...eddC4 to: CrowdfundingContract.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x992...91144

[vm] from: 0x5B3...eddC4 to: DonorContract.(constructor) value: 0 wei data: 0x608...39138 logs: 0 hash: 0x8dc...5f0db

TRANSACTIONS:

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Paris

Transaction: **createCampaign** (string, uint256, uint256)

Parameters: "Clean Water Fund", 5000, 30, 0 wei

Gas Limit: auto, 0

Transact

Balance: 0 ETH

```
8      uint256, string memory, uint256, uint256, address, bool
9    );
10  }
11
12  contract DonorContract {
13
14    struct Donation {
15      uint256 donationId;
16      uint256 campaignId;
17      address donor;
18      uint256 amount;
19      uint256 timestamp;
20    }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

[vm] from: 0x583...eddC4 to: CrowdfundingContract.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x992...91144 Debug

[vm] from: 0x583...eddC4 to: DonorContract.(constructor) value: 0 wei data: 0x608...39138 logs: 0 hash: 0x8dc...5f0db Debug

[vm] from: 0x583...eddC4 to: CrowdfundingContract.createCampaign(string,uint256,uint256) 0xd91...39138 value: 0 wei data: 0x302...00000 logs: 1 hash: 0x58d...ef6f6 Debug

DONATIONS

Low level interaction

Transaction: **donate** (uint256, uint256)

Parameters: 1, 500, 0 wei, 0 gas

Transact

Balance: 0 ETH

```
18      uint256 amount;
19      uint256 timestamp;
20  }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x583...eddC4 to: CrowdfundingContract.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x992...91144 Debug

[vm] from: 0x583...eddC4 to: DonorContract.(constructor) value: 0 wei data: 0x608...39138 logs: 0 hash: 0x8dc...5f0db Debug

[vm] from: 0x583...eddC4 to: CrowdfundingContract.createCampaign(string,uint256,uint256) 0xd91...39138 value: 0 wei data: 0x302...00000 logs: 1 hash: 0x58d...ef6f6 Debug

[vm] from: 0x583...eddC4 to: CrowdfundingContract.createCampaign(string,uint256,uint256) 0xd91...39138 value: 0 wei data: 0x302...00000 logs: 1 hash: 0xa8a...d65e7 Debug

[vm] from: 0x583...eddC4 to: DonorContract.donate(uint256,uint256) 0xd8b...33fa8 value: 0 wei data: 0x0cd...001f4 logs: 2 hash: 0x349...b93d0 Debug

Low level interaction

Transaction: **getDonation** (uint256)

Parameters: 1

Call

Balance: 0 ETH

```
18      uint256 amount;
19      uint256 timestamp;
20  }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x583...eddC4 to: CrowdfundingContract.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x992...91144 Debug

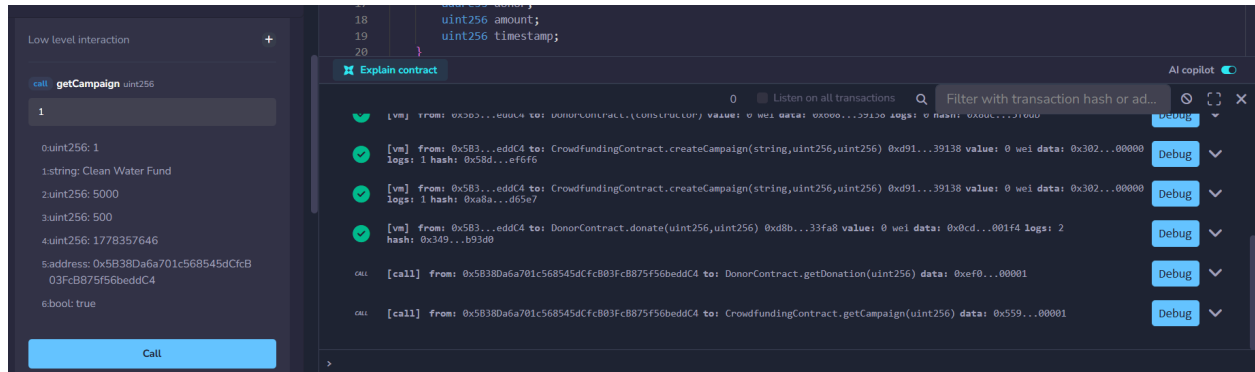
[vm] from: 0x583...eddC4 to: DonorContract.(constructor) value: 0 wei data: 0x608...39138 logs: 0 hash: 0x8dc...5f0db Debug

[vm] from: 0x583...eddC4 to: CrowdfundingContract.createCampaign(string,uint256,uint256) 0xd91...39138 value: 0 wei data: 0x302...00000 logs: 1 hash: 0x58d...ef6f6 Debug

[vm] from: 0x583...eddC4 to: CrowdfundingContract.createCampaign(string,uint256,uint256) 0xd91...39138 value: 0 wei data: 0x302...00000 logs: 1 hash: 0xa8a...d65e7 Debug

[vm] from: 0x583...eddC4 to: DonorContract.donate(uint256,uint256) 0xd8b...33fa8 value: 0 wei data: 0x0cd...001f4 logs: 2 hash: 0x349...b93d0 Debug

[call] from: 0x58380a6a701c568545dcfc803fc875f56beddC4 to: DonorContract.getDonation(uint256) data: 0xf0f...00001 Debug



CONCLUSION:

This experiment helped in understanding how smart contracts can automate and secure a crowdfunding platform on the Ethereum blockchain. Using Solidity and Remix IDE, two contracts — `CrowdfundingContract` and `DonorContract` — were designed, deployed, and tested. The `CrowdfundingContract` manages campaign creation and tracks donations, while the `DonorContract` handles donor interactions. Together they demonstrate a trustless, transparent, and intermediary-free fundraising system.